Authentication Service

Emre Banri - 33672

1. Tools Used

ChatGPT (OpenAI): GPT model, accessed through the ChatGPT web/app interface.

2. How AI Was Used in This Project

For the Authentication Service I used ChatGPT mainly as a tutor, debugging assistant, and checklist helper. I used it to understand the authentication flow, to troubleshoot local environment problems, and to make sure that the service was ready for submission. I did not rely on AI output blindly: I tested the service locally, checked the actual error messages, and verified the final behavior with Docker Compose, pytest, and manual endpoint tests.

The Authentication Service was responsible for user registration, login, password hashing, JWT token creation and validation, protected endpoints, role checking, PostgreSQL integration, Docker Compose integration, and basic testing.

a. Conceptual Explanation

Example Prompt 1: Understanding the authentication flow

Before working on the service, I used ChatGPT to understand how the main parts of an authentication service fit together. I wanted to be able to explain the flow during the technical discussion, not only run the code.

Prompt

```
Explain the basic flow of a FastAPI authentication service with
registration, login, hashed passwords, JWT tokens, and protected
endpoints.
```

ChatGPT response

```
A registration endpoint creates a new user and stores a hashed
password, not the plain password.

A login endpoint receives the email and password, finds the user,
verifies the password against the stored hash, and returns a signed
JWT token when the credentials are correct.

Protected endpoints read the Bearer token from the Authorization
header, verify the token, and use the token data to identify the
current user.

For role-based access, the service first checks that the token is
valid and then checks whether the user has the required role, for
example admin.
```

I used this explanation to understand the structure of my Auth Service. The main flow I followed was: register user, hash password, login user, create token, verify token, and check role for admin-only behavior.

b. Docker and Environment Setup

Example Prompt 2: Docker Compose and environment checks

While checking the backend locally, I needed to make sure that the Auth Service used the correct PostgreSQL database settings, JWT configuration, service ports, and environment files. I also wanted to avoid committing real secrets to the repository.

Prompt

The Auth Service runs with PostgreSQL and JWT settings in Docker Compose. How can I check that DATABASE_URL, JWT_SECRET_KEY, ports, and local environment files are handled correctly without committing real secrets?

ChatGPT response

Check that the Docker Compose service names and ports match the Auth Service configuration.

Use .env for real local values and .env.example only for safe placeholder values.

Make sure .env and other local files are ignored by git.

Run docker compose ps to verify the services are healthy, then test the main auth endpoints manually.

I used this guidance to check the Docker Compose setup, the Auth Service configuration, and the ignored local files. I verified that .env and local.properties were ignored by git, while .env.example only contained placeholder values. This made the project safer to submit and easier for others to configure locally.

c. Docker, Git, and Submission Readiness Checks

Example Prompt 3: Checking project readiness before submission

Before submission, I used ChatGPT to organize the final checks. This helped me avoid missing simple but important things, such as accidentally committing local environment files.

Prompt

```
Give me a short checklist to verify that my authentication service and
the project setup are ready before submission. Include Docker, tests,
git status, and ignored environment files.
```

ChatGPT response

```
Check that Docker Compose starts the services and that the containers
are healthy.

Run the Auth Service tests and any available tests from other
services.

Manually test the main auth flow: register, login, use the JWT token
for a protected endpoint, and check that a normal user cannot access
an admin-only endpoint.

Run git status to confirm the working tree is clean.

Use git check-ignore for .env and local.properties to make sure real
secrets and local paths are not committed.
```

I used this as a checklist and verified the project locally. I checked Docker Compose health, ran the Auth Service tests, confirmed that .env and local.properties were ignored by git, and made sure that the repository was clean before submission work continued.

d. Testing the Auth Endpoints

Example Prompt 4: Manual endpoint verification

After the service was running, I wanted to verify the real behavior of the endpoints instead of only assuming that the code worked.

Prompt

```
How can I manually test a FastAPI authentication service from
PowerShell? I want to test register, login, /auth/me, and an admin-
check endpoint.
```

ChatGPT response

```
Create a test email and password, then send a POST request to the
register endpoint.

Use the same credentials to send a POST request to the login endpoint
and store the returned access token.

Send a request to /auth/me with the Authorization header set to Bearer
<token>.

Finally, call the admin-check endpoint with the same token. If the
user has a normal user role, a 403 response is expected.
```

I manually tested registration, login, the /auth/me protected endpoint, and the /auth/admin-check endpoint. A normal user received 403 on the admin check, which was the expected behavior. This helped me understand the difference between authentication and authorization.

e. Documentation Support

Example Prompt 5: README and contribution wording

I also used ChatGPT to help phrase my contribution clearly and to keep the documentation short and understandable.

Prompt

```
Help me write a short README contribution description for my
Authentication Service. It uses FastAPI, PostgreSQL, password hashing,
JWT login flow, protected endpoints, role checking, Docker Compose
integration, and tests.
```

ChatGPT response

```
Authentication Service: FastAPI service for user registration and
login. The service stores hashed passwords in PostgreSQL, creates JWT
tokens after successful login, protects user endpoints with token
validation, checks roles for admin-only behavior, and runs inside
Docker Compose with basic tests.
```

I adapted this wording for the project documentation and made sure it matched what the service actually did. I also checked that example environment files used placeholders instead of real secret values.

3. Reflection

Using AI helped me most with understanding the authentication flow and checking the project setup. The most useful part was being able to ask why an error happened and what I should verify next, instead of guessing randomly.

The main thing I learned is that AI output still has to be verified. I could not just trust an answer and move on. I had to run the commands myself, read the actual terminal output, test the endpoints, and confirm that the behavior matched the expected result. For example, I confirmed the auth flow manually by registering a user, logging in, using the JWT token for /auth/me, and checking that a non-admin user received 403 for /auth/admin-check.

For the technical discussion, I can explain the core idea of my service: registration stores a user with a hashed password, login verifies the password and returns a JWT token, protected endpoints verify the token, and the admin-check endpoint also checks the user role.

Overall, AI was useful as a tutor, debugging assistant, and documentation helper. It helped me move faster and understand mistakes, but the final responsibility was still mine. I had to test the service, verify the Docker setup, check git status, and make sure that the project was ready for submission.